A Bit-Exact, Processor-Free FPGA Adaptive Noise Canceller: Fair Comparison of LMS, NLMS and Divider-Free NLMS-Shift on Cyclone IV E

Satvik Garg[a], Aditya Gupta[a], Jaffino G[a,*]

[a]*School of Electronics Engineering (SENSE), Vellore Institute of Technology, Vellore, India*

Abstract

Adaptive noise cancellation with a reference input is a classic application of the least-mean-square (LMS) family, and small FPGAs can run it at audio rates without a processor. Published FPGA implementations, however, rarely compare algorithm variants under equal tuning effort, and few show that the hardware reproduces the reference model. This paper presents ULLAP, a parameterisable 12-bit, 32-tap noise canceller for the Terasic DE2-115 (Cyclone IV E) with a WM8731 codec, supporting LMS, NLMS with a restoring divider, and NLMS-Shift, in which the normalisation is a leading-one detection and a barrel shift. All three share one time-multiplexed multiplier and are verified bit-exact against an integer model in 86 regression tests, including the complete codec interface. With step sizes tuned on held-out seeds, NLMS-Shift matches NLMS within the 95% confidence interval on stationary and amplitude-step noise (22.37 and 24.33 dB final noise reduction over 20 seeds) and is about 2 dB better than LMS on amplitude steps, while needing 72 instead of 130 busy cycles per sample and 16% fewer register-transfer-level bit toggles than NLMS. On real recorded noise and speech at 0 dB input SNR, the three algorithms remove 11.7–14.9 dB of noise and raise STOI from 0.69–0.78 to 0.95–0.98 and narrow-band PESQ from 1.4–1.5 to 2.3–2.8, with NLMS 0.4–0.6 dB ahead of LMS and NLMS-Shift between them. A word-length study at the tuned settings shows 11 bits is the shortest word within 3 dB of floating point and 12 bits keeps the loss below 0.4 dB

*Corresponding author. [TBD: e-mail and author order to be confirmed by the authors]

on broadband noise. All variants close timing at 50 MHz. On the board, Signal Tap captures of real codec samples show every one of 351,723 output samples of the three algorithms equal to the integer model's, and the measured sample rate, latency and busy cycles match the RTL. [TBD: Measured on-board noise reduction, after board session 2.]

Keywords: adaptive noise cancellation, NLMS, divider-free normalisation, FPGA, fixed-point arithmetic, bit-exact verification

---

1. Introduction

Adaptive noise cancellation (ANC) estimates the noise that reaches a primary sensor from a separate reference sensor and subtracts it [1]. The LMS algorithm and its normalised form (NLMS) remain the standard choice for this task because of their low cost and robustness. NLMS makes the adaptation speed independent of the input level, but it needs a division per sample, which is expensive in hardware. Replacing that division by a power-of-two approximation of the input power is an old idea [2], and quantised-normaliser and lookup-table reciprocals have been proposed since [3, 4].

Two questions are left open by the FPGA literature on these filters. First, variants are usually compared with different tuning effort, so a reported advantage can come from an under-tuned baseline; this happened in an earlier version of this work, where a 5.7 dB advantage of the divider-free variant over NLMS disappeared after NLMS was regularised fairly. Second, hardware results are rarely tied to the reference model: resource and timing reports show that a design fits, not that it computes what the simulation claims.

This paper addresses both on a low-cost platform and makes the following contributions:

1. A single serial core that implements LMS, NLMS (restoring divider) and NLMS-Shift with one shared multiplier, so the three are compared on identical datapath, word length and filter length.
2. A fair comparison: every algorithm's step size and regulariser are chosen on held-out seeds with the same criterion, and results are reported with 20-seed 95% confidence intervals, on synthetic scenarios and on real recorded noise and speech.
3. A bit-exact verification chain from a Python integer model to the RTL and the board wrapper (86 regression tests including codec, freeze and

reset behaviour), and on-board captures of real codec samples in which every output sample of all three algorithms equals the model's.

4. A reproducible word-length study and a complete processor-free audio system that closes timing at 50 MHz on Cyclone IV E, released with all scripts and data.

The algorithmic idea behind NLMS-Shift is not claimed as new; Section 2 positions it.

2. Related work

Garrett et al. [2] replaced the NLMS division in a decision-feedback equaliser by power-of-two operations obtained from leading-zero counts of the error and the filter energy; NLMS-Shift is the real-valued, error-exact special case of that approach. Zhao et al. [3] quantised the normaliser of a variable-regularised LMS beamformer to a few levels implemented with canonical signed digits and report at least 21% fewer LUTs and 26.85% higher throughput than a divider design on Virtex-7 (22-bit words, eight sensors). Huang et al. [4] replaced the divider by a 6-bit-input, 4-bit-output reciprocal table; on the same DE2-115 board used here, their 16-tap, 16-bit fully parallel NLMS FIR filter uses 11,276 logic elements, runs at 91 MHz with 11 cycles per sample, and reaches 10.52 nJ per sample against 18.60 nJ for a 4-bit SRT divider. Huang and Yao [5] proposed a binary reciprocal approximation built from a leading-one detector and a barrel shifter for a fifth-order NLMS IIR filter, again on the DE2-115, and report 5.07 nJ per sample against 8.33 nJ for a 15-bit SRT divider. The base work of this project [6] simulated an LMS feedback suppressor in MATLAB and Vivado without numeric results. None of these works compares LMS, a true divider and a shift normaliser on one core with equal tuning and statistical tests, and few test a complete audio system with a codec on a board. Compared with these, ULLAP's contribution is the equal-effort three-way comparison on one core, the bit-exact chain to hardware, and the evaluation on real recordings.

3. Algorithms and fixed-point model

With reference vector $\mathbf{x}[n] = [x[n], \ldots, x[n-L+1]]^T$, weights $\mathbf{w}[n]$ and primary input $d[n]$, all three algorithms compute

$$\begin{aligned} y[n] = \mathbf{w}^T[n]\mathbf{x}[n], \qquad e[n] = d[n] - y[n], \\ \mathbf{w}[n+1] = \mathbf{w}[n] + \mu[n]\, e[n]\, \mathbf{x}[n], \end{aligned} \tag{1}$$

with $\mu[n] = \mu$ for LMS and $\mu[n] = \mu/(P[n] + \epsilon)$, $P[n] = \|\mathbf{x}[n]\|^2$, for NLMS. NLMS-Shift replaces the division by a shift:

$$\mu[n] = 2^{-(\lfloor \log_2(P[n]+\epsilon) \rfloor + 1 + K)}, \tag{2}$$

so that $\mu/(2(P + \epsilon)) < \mu[n] \leq \mu/(P + \epsilon)$ with $\mu = 2^{-K}$: the effective step lies within a factor of two below the NLMS step, which keeps the NLMS stability bound. In hardware the exponent is the position of the leading one of $P[n] + \epsilon$, found by a priority encoder, and the update is a barrel shift of $e[n]x[n - i]$.

All signals and weights are signed $Q1.(W - 1)$ with $W = 12$; the accumulator has $2W + \lceil \log_2 L \rceil$ bits; every narrowing point saturates and raises a sticky flag; rounding is to nearest. The power $P[n]$ is updated incrementally by adding $x^2[n]$ and subtracting $x^2[n - L]$. The integer model (`py/fixedpoint.py`, `py/rtl_model.py`) mirrors these widths exactly and generates the golden vectors.

4. Hardware architecture

4.1. Core

The core (`rtl/anc_core.v`, vendor-neutral Verilog) processes one sample per `in_valid` strobe with a single multiplier pipelined in three stages: L cycles for the filter sum, then the error, then L cycles of weight updates. NLMS adds a sequential restoring divider for the gain; NLMS-Shift replaces it by a leading-one detector and a shift-amount register. A `freeze` input holds the weights while filtering continues. The input-to-output latency is $L + 3$ cycles for LMS and $L + 5$ for NLMS and NLMS-Shift (35 and 37 cycles at $L = 32$), and at the tuned settings the core is busy for 73, 130 and 72 cycles per sample respectively; the NLMS divider runs one iteration per dividend bit, and the dividend widens with the step-size constant, so its cycle count depends on μ.

4.2. Board wrapper

The DE2-115 wrapper contains an I^2C engine that configures the WM8731 at reset (11 registers, acknowledge-checked), an I^2S receiver and transmitter clocked by the 50 MHz system clock with the codec's bit and word clocks sampled as data, two-flop synchronisers on the reset and switches, a frame-latched bypass multiplexer, an overrun flag, and four registered test points

on the GPIO header. The codec runs as master from a 12.5 MHz clock with the sample-rate register set to 1536 fs, giving 8.138 kHz, the rate of the simulation study. No processor or vendor IP is used.

5. Verification

Table 1 summarises the verification. The core testbench compares every $y[n]$ and $e[n]$, the final weights, the sticky flags and the latency with golden vectors for 68 configurations (W from 8 to 31, L from 2 to 32, saturation corner cases, freeze patterns). The top-level testbench uses a behavioural WM8731 model that checks every I^2C write and follows the programmed sample rate, and runs tests T1–T10: configuration, NACK handling, bypass, end-to-end cancellation against golden vectors, freeze, reset, flags, clocks, a 45-point skew sweep, the 8 kHz rate and the GPIO timing. Mutation tests confirm the testbenches fail on injected faults. Verilator `-Wall` lint leaves no unwaived warning.

Table 1: Verification summary. RTL suites pass in both Icarus Verilog and Questa (`results/metrics/iverilog_regression_summary.txt`, `results/local_run_da0b6f4/`).

Suite	Tests	Result
Core golden vectors (`tb_anc_core`)	68	all pass, bit-exact
Protocol corner cases	4	pass
Freeze hold/toggle variants	3	pass
Top level with codec model (T1–T10)	11	pass
Python unit tests	46	pass
Verilator lint (`-Wall`)	–	0 unwaived
On-board Signal Tap captures (T9)	12	all bit-exact

6. Experimental set-up

Synthetic scenarios follow the reference model $x = v$, $d = s + h * v + n_s$ at 8 kHz with 16,000 samples: stationary coloured noise, amplitude steps ($\times 3$ then $\times 0.25$), an abrupt path switch at the midpoint, and a recorded chord as the noise source. Step sizes and regularisers were selected on seeds 101–103 by maximising the mean final noise reduction over stationary, amplitude-step and path-switch scenarios with zero overflow: LMS $\mu = 0.25$; NLMS

$\mu = 0.25$, $\epsilon = 1$; NLMS-Shift $K = 3$, $\epsilon = 0.25$. Results are reported on seeds 1–20. The final noise reduction is $\mathrm{NR} = 10 \log_{10} \sum(d - s)^2 / \sum(e - s)^2$ over the last 25% of samples.

For real recordings, four noise files (air conditioner, vacuum cleaner, copy machine, babble) and ten utterances from MS-SNSD [7] were resampled to 8 kHz; each seed draws a noise segment, an utterance and a random 16-tap acoustic path, at 0 dB input SNR, with and without speech leakage into the reference at −20 dB. Noise reduction, STOI [8] and narrow-band PESQ [9] are computed after a 2 s convergence window.

7. Results

7.1. Synthetic scenarios

Table 2: Final noise reduction, dB, mean [95% CI] over seeds 1–20, $W = 12$, $L = 32$ (`results/paper_numbers.csv`).

Scenario	LMS	NLMS	NLMS-Shift
Stationary	22.39 [22.12, 22.66]	22.28 [22.01, 22.54]	22.37 [22.11, 22.63]
Amplitude steps	22.36 [21.72, 23.00]	24.16 [23.58, 24.73]	24.33 [23.88, 24.78]
Path switch	18.95 [18.81, 19.09]	18.53 [18.39, 18.67]	18.91 [18.76, 19.06]
Recorded chord	5.69 [5.66, 5.73]	5.73 [5.69, 5.76]	5.46 [5.42, 5.50]

Table 2 shows that NLMS-Shift ties NLMS on stationary noise and amplitude steps and ties LMS on stationary noise and the path switch. Both normalised algorithms are about 2 dB better than LMS on amplitude steps, where LMS's fixed step is either too slow after the drop or too noisy before it: no LMS step size reaches 22 dB on stationary noise and 24 dB on amplitude steps at once, while both NLMS variants do (Fig. 1). On the recorded chord NLMS-Shift is 0.2–0.3 dB worse. Paired per-seed tests (Table 3) make "tie" precise: every NLMS-Shift versus NLMS difference is inside a ±0.5 dB equivalence margin (TOST, $p < 0.05$), although some are statistically non-zero, for example +0.38 dB on the path switch; the 1.8–2.0 dB advantage of the normalised algorithms over LMS on amplitude steps is significant ($p < 10^{-7}$).

Figures 2–4 show single runs (seed 1) at the fair-tuned settings. Fig. 2 gives the running noise reduction in each scenario, Fig. 3 the residual error power against the input noise power for stationary noise, and Fig. 4 the response to the amplitude steps: after the drop at 1.2 s LMS settles about 3 dB below both normalised algorithms, consistent with Table 2.

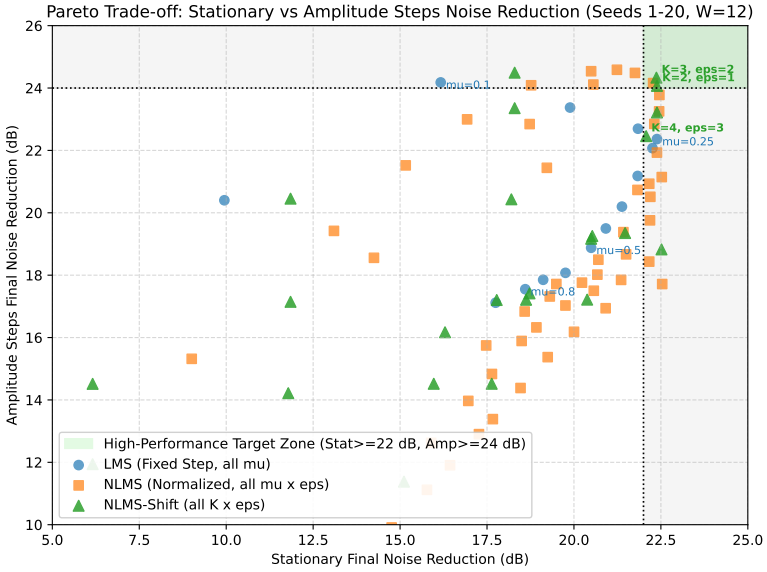


Figure 1: Stationary versus amplitude-step noise reduction across step sizes (seeds 1–20).

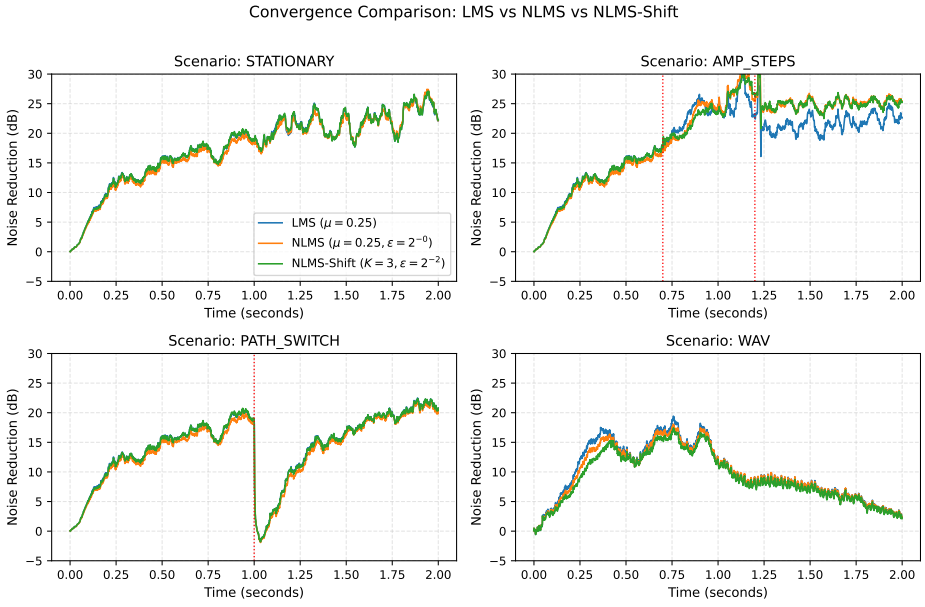


Figure 2: Running noise reduction in the four synthetic scenarios (seed 1, $W = 12$, $L = 32$, fair-tuned settings). Dotted lines mark the amplitude steps and the path switch.

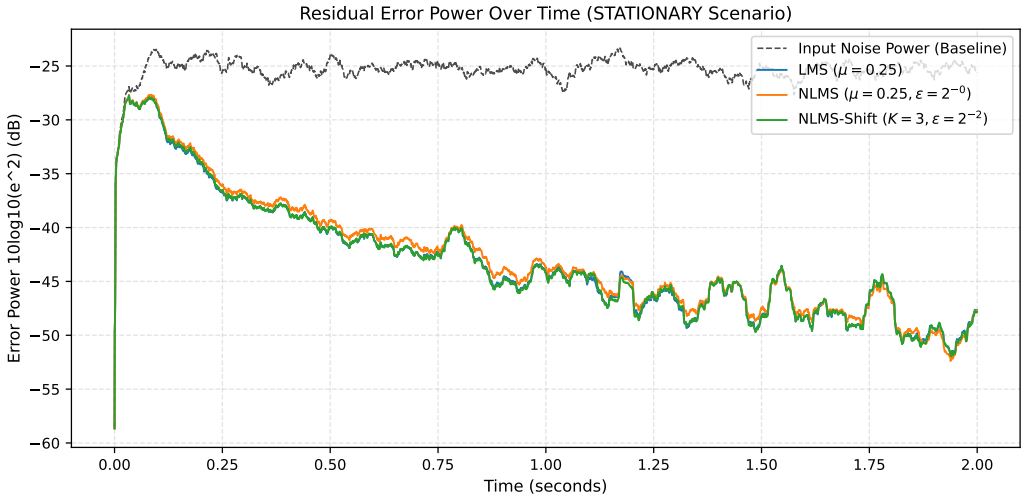


Figure 3: Residual error power over time against the input noise power, stationary scenario (seed 1).

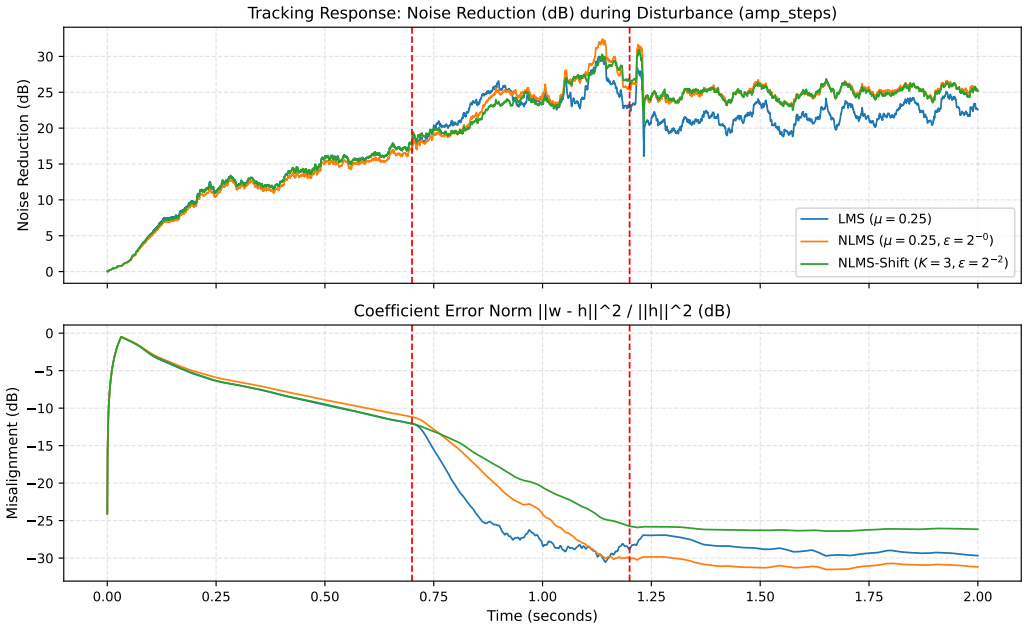


Figure 4: Tracking of the amplitude-step disturbance (seed 1): noise reduction and coefficient misalignment.

Table 3: Paired differences in final NR over seeds 1–20 (`results/metrics/paired_stats.csv`).

Scenario	Pair	Mean diff. (dB)	95% CI	p (paired t)	TOST ±0.5 dB
Stationary	NLMS-Shift − NLMS	+0.10	[+0.01, +0.18]	0.028	equivalent
Stationary	NLMS-Shift − LMS	-0.02	[-0.07, +0.04]	0.58	equivalent
Stationary	NLMS − LMS	-0.11	[-0.18, -0.04]	0.0038	equivalent
Amplitude steps	NLMS-Shift − NLMS	+0.17	[-0.12, +0.47]	0.23	equivalent
Amplitude steps	NLMS-Shift − LMS	+1.97	[+1.51, +2.43]	2.8e-08	not equiv.
Amplitude steps	NLMS − LMS	+1.79	[+1.38, +2.21]	2.5e-08	not equiv.
Path switch	NLMS-Shift − NLMS	+0.38	[+0.30, +0.47]	1.9e-08	equivalent
Path switch	NLMS-Shift − LMS	-0.04	[-0.07, -0.01]	0.018	equivalent
Path switch	NLMS − LMS	-0.42	[-0.50, -0.34]	1.1e-09	equivalent
Recorded chord	NLMS-Shift − NLMS	-0.27	[-0.30, -0.24]	7.7e-14	equivalent
Recorded chord	NLMS-Shift − LMS	-0.24	[-0.26, -0.22]	2.6e-16	equivalent
Recorded chord	NLMS − LMS	+0.03	[+0.00, +0.06]	0.036	equivalent

7.2. Real noise and speech

Table 4: Real recorded noise and speech (MS-SNSD), fixed point $W = 12$, mean ± 95% CI over 20 seeds, no speech leakage into the reference; STOI and PESQ of e against s, with the unprocessed input in brackets (`results/metrics/real_audio_summary.csv`).

Noise	Metric	LMS	NLMS	NLMS-Shift
Air conditioner	NR (dB)	11.65 ± 0.75	12.04 ± 0.82	11.89 ± 0.80
	STOI (input 0.784)	0.948	0.947	0.948
	PESQ (input 1.54)	2.34	2.30	2.33
Vacuum cleaner	NR (dB)	14.32 ± 0.32	14.90 ± 0.33	14.56 ± 0.35
	STOI (input 0.730)	0.982	0.983	0.982
	PESQ (input 1.39)	2.81	2.81	2.81
Copy machine	NR (dB)	14.02 ± 0.38	14.63 ± 0.34	14.39 ± 0.32
	STOI (input 0.694)	0.977	0.978	0.977
	PESQ (input 1.36)	2.74	2.75	2.73
Babble	NR (dB)	11.95 ± 0.42	12.43 ± 0.49	12.27 ± 0.46
	STOI (input 0.719)	0.956	0.955	0.955
	PESQ (input 1.48)	2.48	2.47	2.47

Table 4 shows that on all four real noises the filter roughly doubles PESQ and brings STOI close to 1, and that the order of the algorithms is the same everywhere: NLMS 0.4–0.6 dB ahead of LMS, NLMS-Shift 0.15–0.35 dB behind NLMS. Real noise is less predictable than the coloured synthetic noise, so the noise reduction is 8–11 dB lower than in Table 2. When −20 dB of the speech leaks into the reference, the filter also partly cancels speech: noise

reduction drops by 2.3–4.6 dB, PESQ by 0.25–0.6, and saturation events appear (18,500 over 19.2 million processed samples, most for the air conditioner), so reference-microphone placement matters more than the choice of algorithm (`paper/tab_real_audio_xt.tex`).

7.3. Word length

At the tuned settings, the worst-scenario loss relative to the floating-point run of the same algorithm is 4.1–5.9 dB at $W = 10$, 0.9–1.2 dB at $W = 11$ and 0.3–0.4 dB at $W = 12$ (Fig. 5); 11 bits is the shortest word within the 3 dB criterion and 12 bits was kept for margin. On the tonal chord recording the fixed-point filter outperforms floating point by up to 3.4 dB at $W = 11$ and loses 1.3 dB at $W = 13$: on this narrow-band signal quantisation changes the effective adaptation, so the chord result is sensitive to the word length and should not decide it.

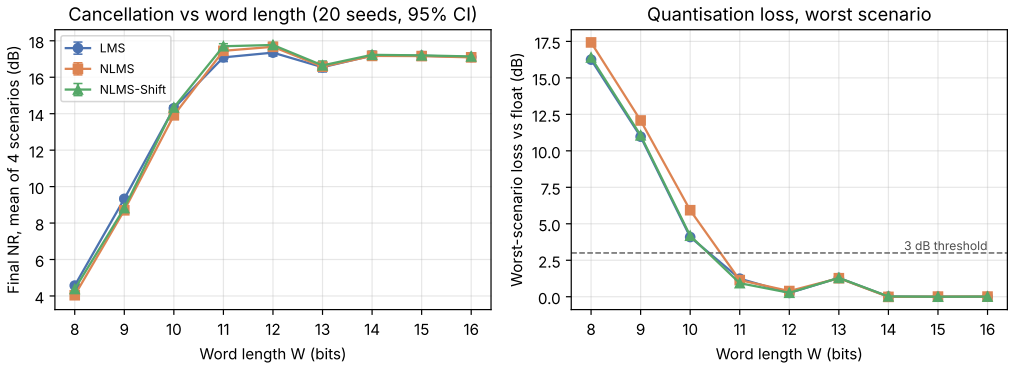


Figure 5: Noise reduction and worst-scenario quantisation loss versus word length (20 seeds).

7.4. Hardware cost

LMS is about 31% smaller than either normalised variant. NLMS-Shift and NLMS use about the same logic (2,610 and 2,626 logic elements), because the divider's area is replaced by the leading-one detector, shifter and wider update path, but NLMS-Shift has 15% fewer registers, needs 45% fewer busy cycles per sample and toggles 16% fewer bits per sample. All three designs meet the 50 MHz constraint with at least +2.953 ns setup slack and +0.401 ns hold slack, with no critical warnings. At 8.138 kHz the busiest

Table 5: Complete board design (core, codec interface, synchronisers, status logic) on Cyclone IV E EP4CE115F29C7, $W = 12$, $L = 32$, Quartus Prime Lite 25.1, commit da0b6f4 (`results/quartus_reports/board_da0b6f4/`, `results/metrics/power_activity.csv`).

	LMS	NLMS	NLMS-Shift
Logic elements	1,792	2,626	2,610
Registers	1,123	1,404	1,186
9-bit multipliers	4	4	4
$F_{\max}$, slow 85 °C (MHz)	70.08	59.75	58.66
Setup slack at 50 MHz, slow 85 °C (ns)	5.731	3.263	2.953
Latency (cycles)	35	37	37
Busy cycles per sample (measured on the board)	73	130	72
RTL bit toggles per sample	5,668	7,146	6,012
Core dynamic power, vectorless (mW)	20.79	24.36	23.77

variant uses 2% of the 6,144-cycle sample period, so a single core could serve many channels or much longer filters.

The power figures are Quartus vectorless estimates with the default 12.5% toggle rate, which Quartus itself rates as low confidence. Core static power (about 98.7 mW) dominates the 169 to 173 mW total. The dynamic estimates rank the designs LMS < NLMS-Shift < NLMS, as the toggle counts do, but a fixed toggle rate mostly tracks area and ignores that the datapath is idle for 98% of each sample period, so these numbers are upper bounds and not energy per sample. A gate-level, activity-annotated estimate is left for future work: the post-fit netlist flattens the hierarchy that the top-level testbench observes, so it needs a netlist-only testbench. For comparison with [4, 5], which use the same board, energy per sample should be reported as dynamic power divided by the sample rate from an activity-annotated Power Analyzer run, which this work does not yet have, so no energy comparison with them is claimed. Their designs are fully parallel and run at megasamples per second, while ULLAP shares one multiplier and runs at 8.138 kHz, so area and energy are compared per tap and per sample, not as raw totals.

7.5. On-board results

The board evidence is recorded by the FPGA itself with Signal Tap, so it does not depend on an external recorder or oscilloscope. One logic-analyser instance stores, once per sample, the two codec samples entering

the core (x and d, register outputs `x_reg` and `d_reg`) and the core output e, with a transitional storage qualifier on `out_valid`, and triggers when the reset button is released; it holds 32,768 samples (4.0 s). A second instance samples the four GPIO test points at 50 MHz. The stimuli are the noise-only versions of the four scenarios, played from a laptop into LINE IN after a 1 kHz calibration tone; the reset is pressed during the tone, so each capture starts from zero weights on real codec data at about -25 dBFS, and no sample in any capture reaches full scale. The analysis script and the raw exports are in the repository (`py/analyze_board_capture.py`, `board_evidence/`).

Bit-exactness. For each of the 12 captures (three algorithms, four scenarios) the integer model of Section 3, started from zero state at the first sample after the reset and fed the captured x and d, reproduces every captured $e[n]$: 351,723 samples in total, 28,672 to 32,768 (3.5 to 4.0 s) per capture, with no mismatch. The fixed-point arithmetic of the silicon is therefore exactly the one evaluated in simulation, also on analogue input with the codec's noise and offsets.

Timing. The GPIO capture gives an `in_valid` period of 6,144 clock cycles (8,138.02 Hz), an `in_valid` to `out_valid` spacing of 38, 36 and 38 cycles for NLMS-Shift, LMS and NLMS (the core latency plus one accept cycle, 0.76, 0.72 and 0.76 μs) and busy times of 72, 73 and 130 cycles, all equal to the RTL simulation of the same parameters. The Signal Tap builds still meet the 50 MHz constraint (setup slack +2.65, +5.68 and +3.78 ns); Table 5 is for the builds without the logic analyser.

Fig. 6 shows one capture: after the tone, the residual falls within about 0.5 s, and when the noise level triples at 2.35 s the normalised filter recovers within a few hundred milliseconds. [TBD: Board session 2: noise reduction on the 2 s seed-1 stimulus (the simulation signal) against the integer model on the same signal, freeze test, 10-minute soak, DAC-word check.]

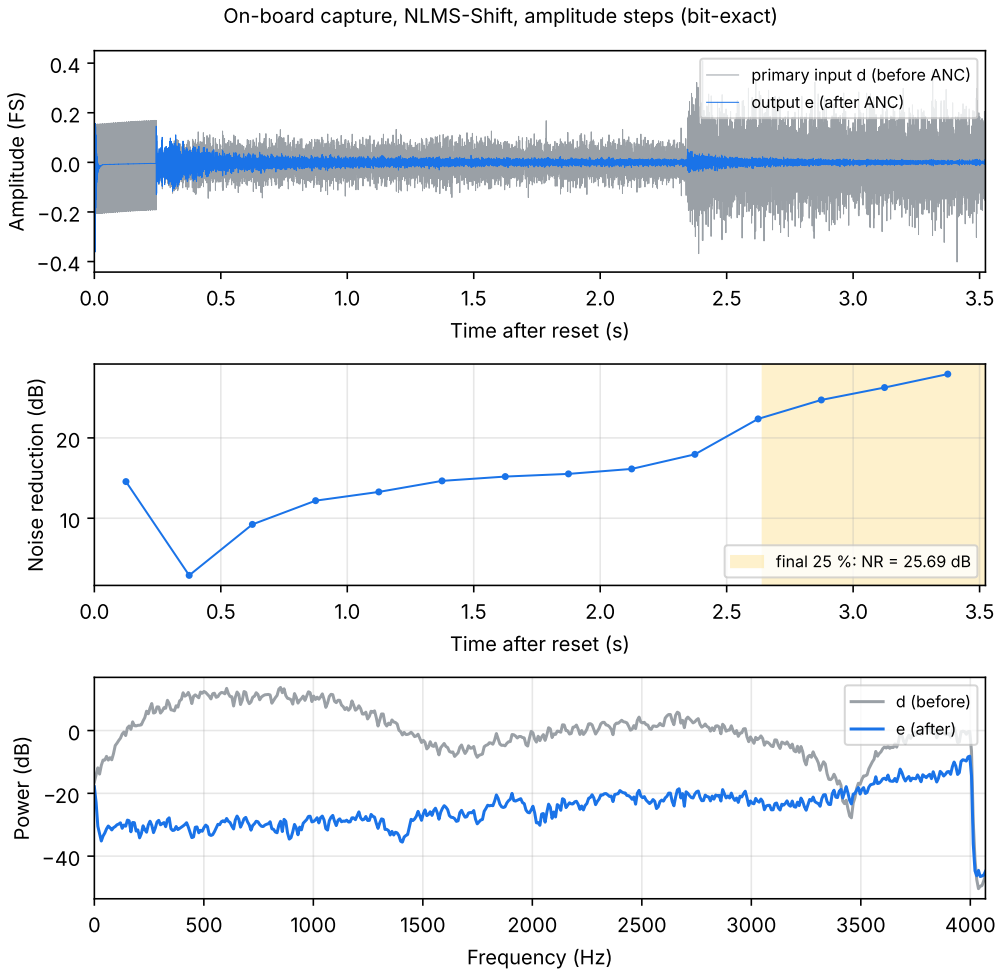


Figure 6: On-board Signal Tap capture, NLMS-Shift, amplitude-step noise: primary input d and output e after the reset (the first 0.25 s is the calibration tone), running noise reduction, and power spectra over the capture. Every output sample equals the integer model's.

8. Discussion and limitations

The comparison supports a narrow conclusion: the divider-free normalisation keeps NLMS's level-independence at about LMS's cycle count, so it is the better choice when the noise level changes and throughput or energy per sample matters, while LMS remains the smallest when the level is steady. It does not show that NLMS-Shift is better than NLMS in cancellation. The acoustic paths are simulated, also for the real recordings; the evaluation does not cover acoustic feedback (the closed-loop howling scenario of the base work), for which reference-based ANC is not the right structure. Power is a switching-activity proxy until gate-level analysis is done. [TBD: Update after the board test.]

9. Conclusion

[TBD: Write after the board results: three to four sentences restating the measured outcome.]

Data and code availability

All RTL, testbenches, scripts and result files are in the project repository [TBD: public URL and Zenodo DOI of the tagged release]. The real-audio study fetches its MS-SNSD files from a pinned commit and checks them by SHA-256.

Declaration of generative AI use

[TBD: Authors: describe the AI tools used for code, analysis, literature search and writing, as required by the journal, and state that the authors reviewed and take responsibility for the content.]

References

[1] B. Widrow, J. R. Glover, J. M. McCool, J. Kaunitz, C. S. Williams, R. H. Hearn, J. R. Zeidler, E. Dong, R. C. Goodlin, Adaptive noise cancelling: principles and applications, Proceedings of the IEEE 63 (12) (1975) 1692–1716.

[2] D. Garrett, C. Nicol, A. Blanksby, C. Howland, A low power normalized-LMS decision feedback equalizer for a wireless packet modem, in: Proc. Int. Symp. Low Power Electronics and Design (ISLPED), 2002.

[3] W. Zhao, J. Q. Lin, S. C. Chan, H. K.-H. So, A division-free and variable-regularized LMS-based generalized sidelobe canceller for adaptive beamforming and its efficient hardware realization, IEEE Access 6 (2018) 64470–64485.

[4] Y.-Z. Huang, C.-Y. Yao, P.-Y. Lee, Speeding up the throughput of an NLMS adaptive filter, in: Proc. IEEE Int. Conf. Consumer Electronics – Taiwan (ICCE-Taiwan), 2024, pp. 221–222.

[5] Y.-Z. Huang, C.-Y. Yao, Enhancing the power efficiency of the NLMS adaptive IIR filter with fast binary reciprocal approximation, Circuits, Systems, and Signal Processing 45 (2026) 7561–7584.

[6] W. S. C. Rodrigo, R. A. P. Buddhika, Real-time FPGA-based audio processing for enhanced acoustic feedback suppression, in: Proc. TIDAC Research Symposium, Colombo, Sri Lanka, 2024.

[7] C. K. A. Reddy, E. Beyrami, J. Pool, R. Cutler, S. Srinivasan, J. Gehrke, A scalable noisy speech dataset and online subjective test framework, in: Proc. Interspeech, 2019, pp. 1816–1820.

[8] C. H. Taal, R. C. Hendriks, R. Heusdens, J. Jensen, An algorithm for intelligibility prediction of time-frequency weighted noisy speech, IEEE Transactions on Audio, Speech, and Language Processing 19 (7) (2011) 2125–2136.

[9] ITU-T, Recommendation P.862: Perceptual evaluation of speech quality (PESQ), International Telecommunication Union (2001).